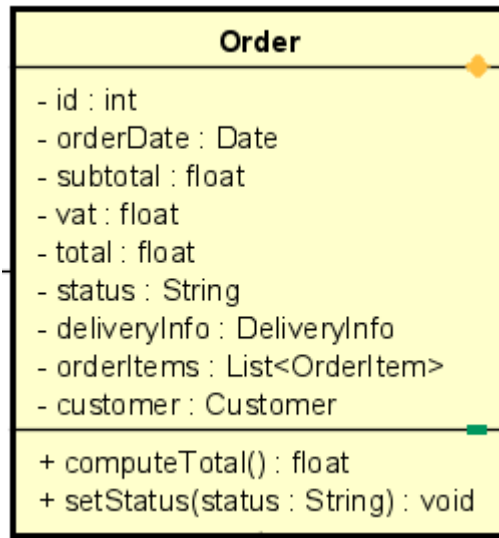


## Design for class “Order”



**Table 1. Attribute Design**

| # | Name         | Data type    | Default value | Description                         |
|---|--------------|--------------|---------------|-------------------------------------|
| 1 | id           | int          | 0             | Unique order ID                     |
| 2 | orderDate    | Date         | current date  | Timestamp when the order is created |
| 3 | subtotal     | float        | 0.0           | Subtotal cost (sum of items)        |
| 4 | tax          | float        | 0.0           | VAT or additional tax               |
| 5 | total        | float        | 0.0           | Total amount after tax              |
| 6 | status       | String       | "CREATED"     | Current order status                |
| 7 | deliveryInfo | DeliveryInfo | null          | Associated delivery info            |
| 8 | orderItems   | List         | empty list    | List of order items                 |
| 9 | customer     | Customer     | null          | Customer placing the order          |

**Table 2. Operation Design**

| # | Name              | Return type | Description / Purpose            |
|---|-------------------|-------------|----------------------------------|
| 1 | computeTotal()    | float       | Calculate subtotal + tax → total |
| 2 | setStatus(status) | void        | Update the order’s status        |

## **Method Design:**

### **1. computeTotal**

- Parameter:
  
- Exception:
  
- Method:
  - subtotal = sum of all item.computeSubTotal().
  - total = subtotal + tax.
  - Return total.

### **2. setStatus**

- Parameter:
  - status: String
  
- Exception:
  
- Method:
  - Assign new status to order.
  - No return value.